

The background of the entire page is a light cream color. It is decorated with several stylized, flat-design illustrations of diverse people in various roles. At the top, from left to right, there is a woman with a stethoscope, a person holding a camera, a baby in a red basket, a man with a beard and a stethoscope, and a woman with glasses holding a pointer. On the left side, there is a person with a paint palette and a woman with a beret. At the bottom, from left to right, there is a woman with a microphone, a person in a construction hard hat, a man in a suit with a microphone, a woman in a cowboy hat, and a police officer. The central text is in a large, bold, dark brown font.

COMMUNITY MANAGEMENT SYSTEM

LAURA BRULÉ, ANAÏS FERREIRA, THALIA GHALI, MITHIA RATSIMBARISON

PROJECT

Goal: connect individuals & communities in a collaborative help network

CORE

- Decentralized links
 - individuals → unidirectional links with descriptions
 - Communities → links to other communities
- Self-governed communities
 - any member can manage their community's information
- Democratic moderation
 - exclusion vote > 50% → member removed

GEOLOCATION

- Users may register GPS coordinates
- System generates OpenstreetMap links
- Detect individuals / communities within 1km

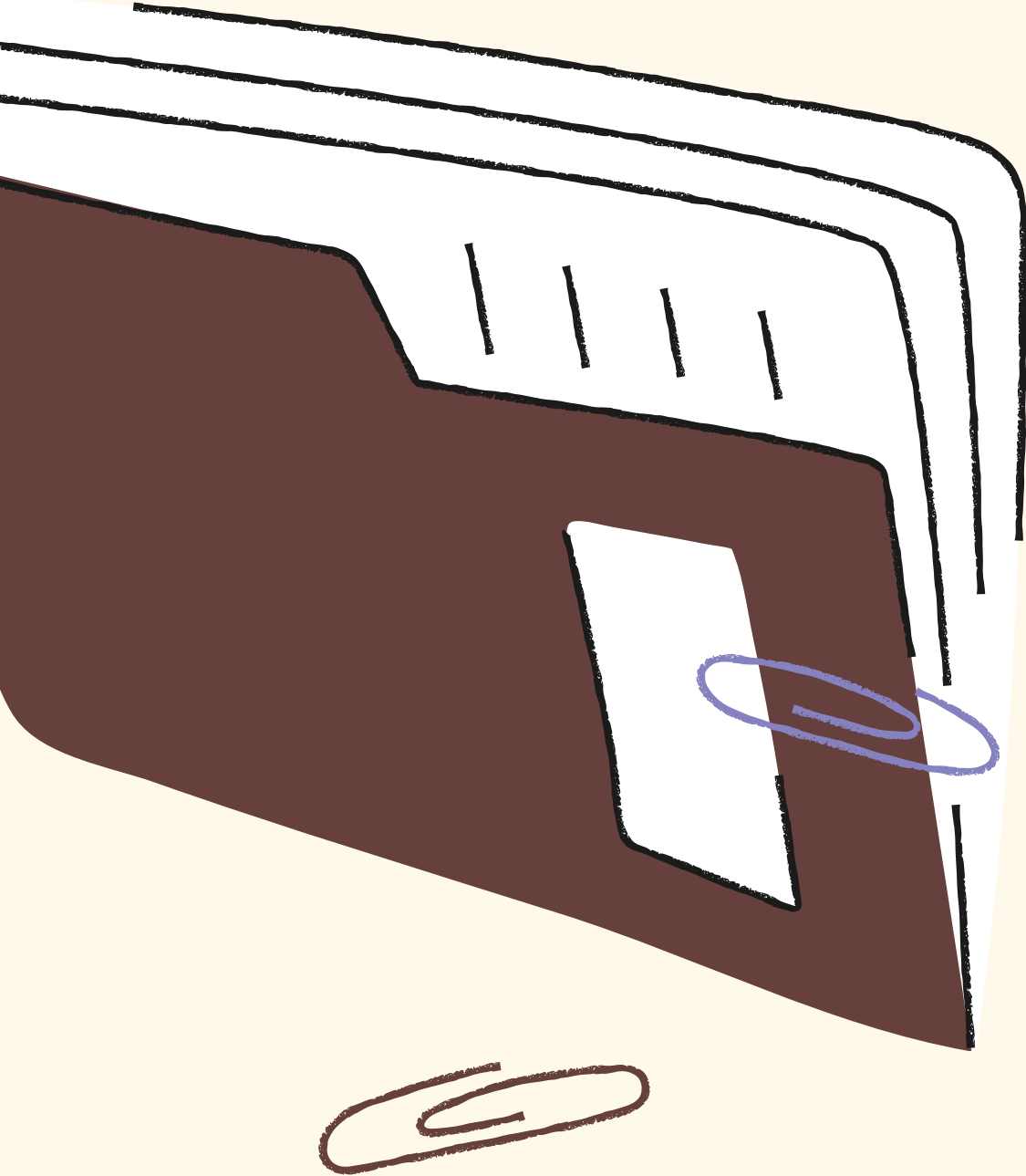
CONTEXT

FUNCTIONAL SCOPE

- Skills & expertise (levels 1-5)
- Services
 - Free
 - Exchange-based
 - Commercial (G1 cryptocurrency)
- Messaging
 - Sender-owned messages
 - Supports threads (reply to another message)

PROJECT FUNCTION

- Build a complete system (DB + Python App + GUI)
- Fully implement all required functionalities
- Respect constraints: PostgreSQL, python UML, documentation, 3 mandatory views)



1 Joint participation in the entire reflection process

**DESIGNING THE LOGICAL MODEL,
STRUCTURING THE SQL SCHEMA,
ORGANIZING THE OVERALL PYTHON SCRIPTS.**

2 Final implementation

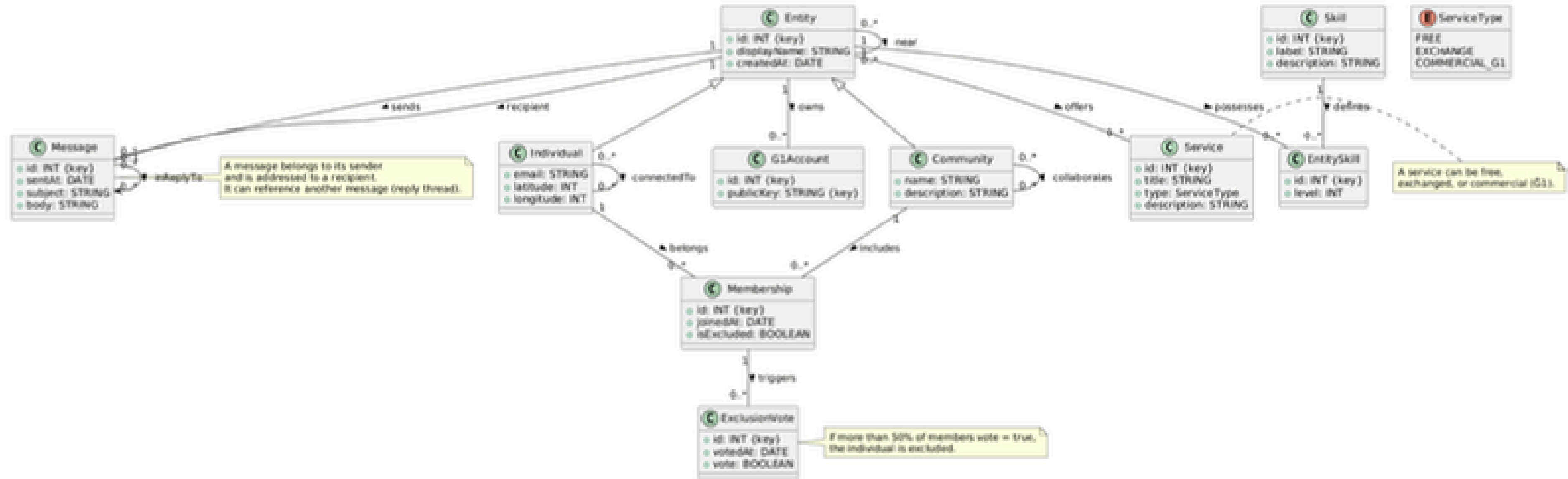
**EACH MEMBER TOOK RESPONSIBILITY FOR ONE OR
SEVERAL SPECIFIC PYTHON FILES,**

**INDIVIDUAL IMPLEMENTATION WHILE REMAINING FULLY
CONSISTENT WITH THE MODEL DEFINED AS A GROUP,**

**CROSS-REVIEWING AND TESTING TO ENSURE THE
QUALITY AND UNIFORMITY OF THE PROJECT.**

repartition

Community Management System - Data Conceptual Model



UML

LOGICAL DATA MODEL

ENTITY (#ID : INT , DISPLAYNAME: STRING, CREATEDAT: DATE, ENTITY_NEAR => ENTITY(ID))

ENTITY_PROXIMITY (#ENTITY1_PROXIMITY => ENTITY (ID), # ENTITY2_PROXIMITY (ENTITY(ID)), DISTANCE_M : INT)

SERVICE(ID INT PRIMARY KEY, TITLE STRING NOT NULL, TYPE STRING NOT NULL CHECK (TYPE IN ('FREE', 'EXCHANGE', 'COMMERCIAL_GI')),DESCRIPTION STRING, ENTITY_ID INT NOT NULL => ENTITY(ID))

ENTITSKILL (#ID : INT, LEVEL : INT, ENTITY_ID : INT NOT NULL => ENTITY, SKILL_ID : INT NOT NULL => SKILL CHECK (LEVEL BETWEEN 1 AND 5))

SKILL (#ID : INT, LABEL : STRING, DESCRIPTION : STRING)

INDIVIDUAL (#ID_INDIVIDUAL => ENTITY(ID), EMAIL : STRING UNIQUE, LATITUDE: INT, LONGITUDE : INT)

COMMUNITY (#ID_COMMUNITY => ENTITY(ID), NAME : STRING UNIQUE, DESCRIPTION: STRING)

VINDIVIDUAL = JOIN(ENTITY, INDIVIDUAL, ENTITY.ID = INDIVIDUAL.ID)

VCOMMUNITY = JOIN(ENTITY, COMMUNITY, ENTITY.ID = COMMUNITY.ID)

VPROXIMITY = JOIN(INDIVIDUAL I1, INDIVIDUAL I2, ENTITY E1, ENTITY E2, I1.ID_INDIVIDUAL = E1.ID AND I2.ID_INDIVIDUAL = E2.ID) WHERE I1.ID_INDIVIDUAL ≠ I2.ID_INDIVIDUAL

COMMUNITY_COLLABORATION (#ID_COMMUNITY1 => COMMUNITY(ID_COMMUNITY), #ID_COMMUNITY2 => COMMUNITY(ID_COMMUNITY))

INDIVIDUAL_CONNEXION (#ID_INDIVIDUAL1 => INDIVIDUAL(ID_INDIVIDUAL), #ID_INDIVIDUAL2 => INDIVIDUAL(ID_INDIVIDUAL))

MEMBERSHIP (# ID : INT, JOINEDAT : DATE, ISEXCLUDED: BOOLEAN, ID_COMMUNITY: INT NOT NULL => COMMUNITY (ID_COMMUNITY), ID_INDIVIDUAL: INT NOT NULL=> INDIVIDUAL(ID_INDIVIDUAL))

EXCLUSIONVOTE (#ID : INT, VOTEDAT : DATE, VOTE : BOOLEAN, MEMBERSHIP_ID : INT NOT NULL => MEMBERSHIP(ID))

GIACCOUNT(#ID : INT, #PUBLICKEY NOT NULL UNIQUE : STRING, ENTITY_ID: INT NOT NULL => ENTITY(ID))

MESSAGE(#ID : INT, SENTAT : DATE, SUBJECT: STRING, BODY: STRING, SENDER_ID : INT NOT NULL => ENTITY(ID) , RECIPIENT_ID : INT NOT NULL => ENTITY(ID) , INREPLYTO_ID : INT NOT NULL => MESSAGE(ID))

CONSTRAINTS

- Only individuals belonging to the same community are allowed to vote against each other.
- Each user must declare their skills along with a proficiency level from 1 to 5.
- The initial constraint was to find an individual located less than 1 km from another, but in our system we are free to choose any maximum distance we want (in km).
- A user can reply to a given message only once.

DATABASE MANAGEMENT

populate_database.py

Fills the database with initial data
→ Inserts sample

clear_database.py

Clears the database
→ Deletes data

db_connection.py

Handles the database connection
→ Opens the connection

MAIN

```
=====
COMMUNITY MANAGEMENT SYSTEM
=====
=== VIEWS ===
1. Community View (my communities)
2. Message View (my messages)
3. Proximity View (geographic proximity)

=== SEARCH ===
4. Search people
5. Search communities
6. Search skills
7. Search services

=== CREATION ===
8. Add a person
9. Add a community
10. Send a message
11. Join a community
12. Add a skill
13. Associate a skill with an entity
14. Add a service
15. Connect two individuals
16. Create a collaboration between communities

0. Quit
=====
Your choice: 
```

- Core Engine
- Functions Called by GUI & User GUI
- Complete List of System Operations

GUI application for Community Management System

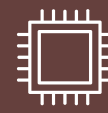


import all the function from the main to work with and the library tkinter

Architecture & Role

Front-End Layer: This file handles display only. It contains zero SQL.

Delegation: Acts as a bridge. It captures user inputs and calls functions imported from main.py (the backend) to execute logic.



UX Structure & Components

Organized via ttk.Notebook to separate workflows: Views, Search, Create. Features a dynamic Treeview that automatically adapts columns to display any data type (Users, Messages, etc.).



Robust Interaction Flow

Buttons trigger "Action methods" that sanitize inputs (type checking, cleaning) before calling the backend. Integrated try/except blocks catch errors to prevent crashes and display user alerts.

KEY FEATURES

User_GUI

Primary user-facing interface

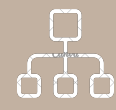
Why it's important:

This interface implements exactly the user interaction model required by the project specification.



Login & User Session

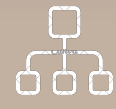
Loads user identity and personal data; all actions executed as the logged-in individual.



Personal Dashboard

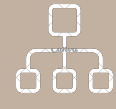
Quick access to:

My Communities • My Skills • My Services • My Messages • My Connections



Community Management

Join/leave communities, view members, explore community lists.



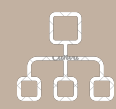
Messaging System

Send messages, reply to threads, view received messages.



Skills & Services

Declare skills (level 1–5), search global skills, create and offer services (Free, Exchange, Commercial Ğ1).



Proximity View

Shows nearby individuals within a chosen distance using the SQL view vProximity.



Governance & Exclusion Votes

View members, cast exclusion votes, compute >50% rule, automatic exclusion.

Jane, Rayan, Tom and Lola want to be part of a community



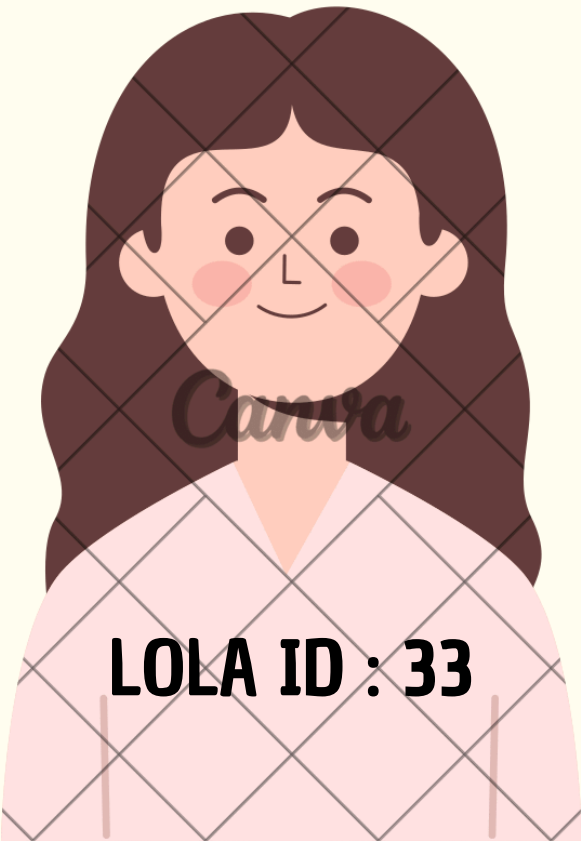
PASTRY: 34

Trompe l'oeil
Icing

Bread
Puff pastry

Choux pastry
Bun

Jam
Fruit tart



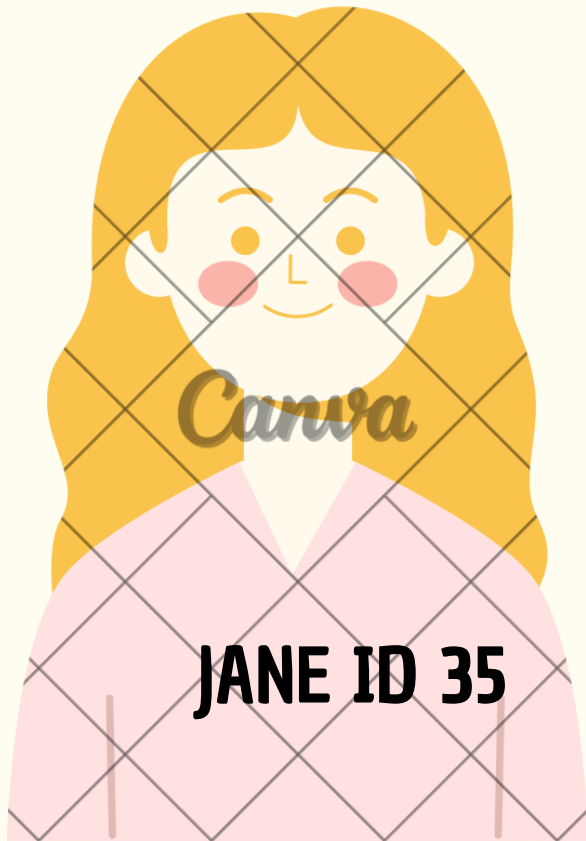
LOLA ID : 33



TOM ID : 32



RAYAN ID 36



JANE ID 35

CONCLUSION & FUTURE WORK

This project allowed us to design a complete system, from the logical model to the SQL schema and Python implementation. We collaborated on the overall structure, then each contributed individually to specific Python files, ensuring consistency through cross-review and testing.

Overall, the project strengthened our understanding of data modeling, constraints, and collaborative development, resulting in a coherent and reliable final system.

For the future work we can create a more intuitive interface, or add another constraint that if we are excluded from a community we cannot join it again.